

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Case Study)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO4-AT2-CASE STUDY
Question	Q1 — Personal Finance Management Package in Python (50 Marks)
Section	Section B — System Design (10 Marks)

Q1. Section B — System Design [10 Marks]: Design the architecture, class model, and data model for the Personal Finance Management package.

Module Architecture

The package is split into five cooperating modules, mirroring the four responsibilities identified in Section A plus a shared exceptions module:

```
finance_pkg/  
|-- exceptions.py      Custom exception hierarchy (FinanceError and subclasses)  
|-- models.py          Transaction and Account data classes  
|-- budget_manager.py  Loads budgets.json, checks spend vs limit  
|-- finance_manager.py Core engine: accounts, transactions, CSV I/O, validation  
|-- reports.py         Reporting & analytics: summaries, breakdowns, text report  
|-- main.py           Wires everything together and runs the package end-to-end
```

Data flows in one direction: raw CSV rows or manual calls enter through FinanceManager, which validates against models.py and exceptions.py and updates Account balances; reports.py then reads only the resulting Transaction list and BudgetManager data to produce analytics — it never touches raw CSV rows or validation logic directly, keeping reporting decoupled from input handling exactly as required by NFR4 (maintainability) from Section A.

Class Model

```
Transaction  
- txn_id, date, txn_type ("Income"/"Expense"), category, amount, description, account  
- signed_amount() -> +amount for Income, -amount for Expense  
- to_row()        -> list form for CSV writing  
Validates date format and amount > 0 in __init__, raising InvalidDateError /  
InvalidAmountError immediately on construction.  
  
Account  
- name, opening_balance, balance  
- apply(transaction) -> updates balance by transaction.signed_amount()  
  
BudgetManager  
- budgets: dict[category -> monthly limit]  
- from_json(path)    -> classmethod, loads budgets.json  
- limit_for(category)  
- check(category, spent) -> {limit, spent, remaining, status}  
  
FinanceManager (the core engine / facade)  
- accounts: dict[name -> Account]  
- transactions: list[Transaction]
```

```

- add_account(name, opening_balance)
- add_transaction(date, type, category, amount, description, account)
  validates category/account/amount/date, checks funds, applies to Account,
  appends to transactions -- single entry point used by BOTH manual calls
  and CSV import, so every transaction is validated identically
- save_transactions_csv(path) / load_transactions_csv(path)
- transactions_in_month(year, month) -> used by the reporting layer

reports.py (free functions operating on a FinanceManager + BudgetManager)
- monthly_summary(manager, year, month)
- category_breakdown(manager, year, month, txn_type)
- budget_vs_actual(manager, budget_manager, year, month)
- savings_rate(manager, year, month)
- top_categories(manager, year, month, n)
- write_monthly_report(...) -> writes the formatted .txt report

```

Exception Hierarchy

```

FinanceError (base)
|-- InvalidAmountError      non-positive transaction amount
|-- InvalidCategoryError    category not in ALL_CATEGORIES
|-- InvalidDateError        date string not in YYYY-MM-DD format
|-- InvalidAccountError     account name not registered
|-- InsufficientFundsError  expense would take balance below zero

```

All five inherit from a common `FinanceError` base so calling code can catch every finance-package error with one except clause when it only cares that **something** went wrong, or catch a specific subclass when it needs to react differently — this mirrors the `BillingError`-style hierarchy pattern, applied here to personal finance.

Data Model (persisted files)

transactions.csv — one row per transaction:

```
txn_id,date,type,category,amount,description,account
```

budgets.json — category to monthly limit map:

```
{ "Food": 6000, "Rent": 15000, "Transport": 3000, ... }
```

Why this design satisfies the Section A requirements

- **FR1/FR2 (multi-account tracking, transaction logging):** Account and Transaction are separate classes; FinanceManager owns both collections and is the only place that mutates them.
- **FR3 (category-based budgeting):** BudgetManager is fully decoupled from FinanceManager and only depends on category names and spend totals it is given — it never reads transactions.csv directly.
- **FR4 (reporting):** reports.py depends only on FinanceManager's public query method (`transactions_in_month`) and BudgetManager's `check()` method, not on internal validation logic.
- **FR5 (extensibility):** adding a new category means adding its name to the category sets in models.py and an entry to budgets.json — no class in finance_manager.py or reports.py needs to change, since none of them hardcode category names.

Conclusion

The design separates validation (models.py + exceptions.py), orchestration (finance_manager.py), configuration (budget_manager.py), and analytics (reports.py) into independent modules connected only through simple, well-defined data (Transaction objects, Account objects, and plain dicts). This directly enables the extensibility and robustness goals from Section A, and is implemented and exercised end-to-end in Sections C, D, and E.